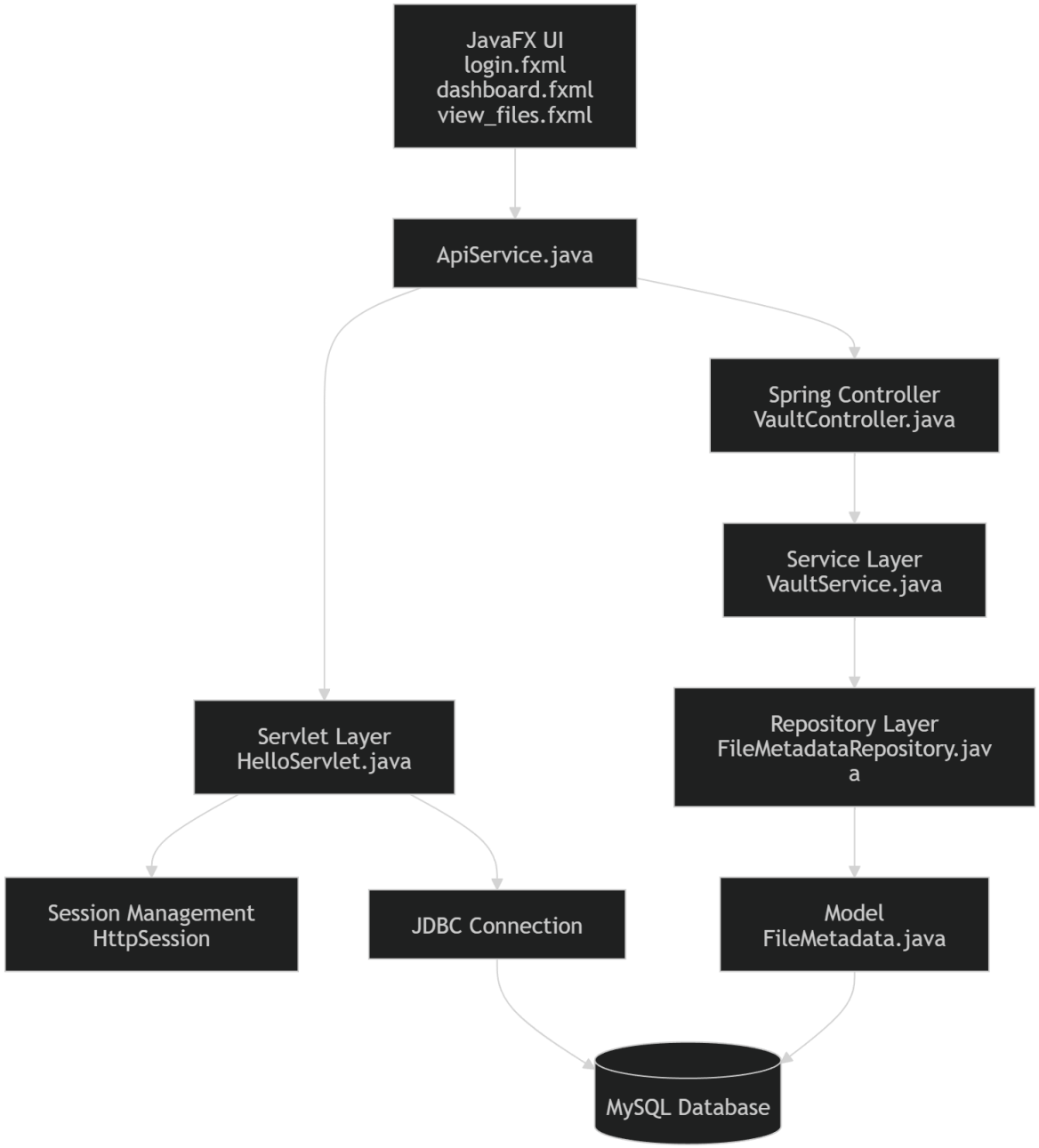
Secure Digital Vault System

Project Description

The Secure Digital Vault System is a Java-based application designed to manage file metadata securely. The front-end is developed using JavaFX, providing a desktop user interface. The backend is built using Spring Boot, which exposes REST APIs and connects to a MySQL database using JDBC (via Hibernate ORM). Additionally, a servlet is implemented to handle HTTP requests, demonstrate session management, and retrieve database records. The system integrates all components to provide a complete client-server architecture.

Project Architecture Diagram (MVC)



MVC Layers with File Names

VIEW (Frontend - JavaFX)

- MainApp.java
- login.fxml
- dashboard.fxml
- view_files.fxml

CONTROLLER

- VaultController.java (Spring Boot REST Controller)
- HelloServlet.java (Servlet Controller)

SERVICE

- VaultService.java

MODEL

- FileMetadata.java

REPOSITORY

- FileMetadataRepository.java

CONFIGURATION

- application.properties
- pom.xml

Modules Description

1. JavaFX GUI Module

Provides user interface for login, dashboard, and viewing file metadata.

2. Backend Module (Spring Boot)

Handles REST API requests and business logic.

3. Database Module (MySQL + JDBC)

Stores file metadata and support CRUD operations.

4. Servlet Module

Implements HttpServlet to process HTTP requests and demonstrate session management.

5. Integration Module

Connects JavaFX frontend with backend and servlet using HTTP communication.

Modules Completed

- JavaFX UI (Login, Dashboard, File View)
- Spring Boot Backend API
- MySQL Database Integration
- Hibernate (JPA) MappingServlet Implementation (HttpServlet)
- Session Management (HttpSession)
- HTML Page Integration
- JavaFX ↔ Servlet Communication

Codes
JavaFX (Frontend)

MainApp.java

```
package com.securevaultclient;

import com.securevaultclient.util.SceneSwitcher;
import javafx.application.Application;
import javafx.stage.Stage;

public class MainApp extends Application {

    private static MainApp instance;
    private Stage primaryStage;
    private String currentUsername;

    @Override
    public void start(Stage stage) {
        instance = this;
        this.primaryStage = stage;
        this.primaryStage.setTitle("Secure Digital Vault");
        this.primaryStage.setResizable(false);
```

```
        SceneSwitcher.switchTo("login");
    }

    public static MainApp getInstance() {
        return instance;
    }

    public Stage getPrimaryStage() {
        return primaryStage;
    }

    public String getCurrentUsername() {
        return currentUsername;
    }

    public void setCurrentUsername(String username) {
        this.currentUsername = username;
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

DashboardController.java

```
package com.securevaultclient.controller;

import com.securevaultclient.MainApp;
import com.securevaultclient.service.ApiService;
import com.securevaultclient.util.AlertUtil;
import com.securevaultclient.util.SceneSwitcher;
import javafx.fxml.FXML;
```

```
import javafx.scene.control.Label;

import javafx.scene.Scene;
import javafx.scene.web.WebView;
import javafx.stage.Stage;

public class DashboardController {

    @FXML
    private Label welcomeLabel;

    @FXML
    public void initialize() {
        String username = MainApp.getInstance().getCurrentUsername();
        welcomeLabel.setText("Welcome, " + username + "!");
    }

//    @FXML
//    private void handleServlet() {
//        String result = ApiService.getInstance().callServlet();
//        AlertUtil.showInfo("Servlet Response", result);
//    }


//    web view for call servlet
    @FXML
    private void handleServlet() {

    String result = ApiService.getInstance().callServlet();

    WebView webView = new WebView();
    webView.getEngine().loadContent(result);
```

```
    Stage stage = new Stage();
    stage.setTitle("Servlet Output");

    Scene scene = new Scene(webView, 600, 400);
    stage.setScene(scene);

    stage.show();
}

    @FXML
    private void handleAddFile() {
        SceneSwitcher.switchTo("add_file");
    }

    @FXML
    private void handleViewFiles() {
        SceneSwitcher.switchTo("view_files");
    }

    @FXML
    private void handleVerifyFile() {
        SceneSwitcher.switchTo("verify");
    }

    @FXML
    private void handleViewAudit() {
        SceneSwitcher.switchTo("audit");
    }

    @FXML
    private void handleLogout() {
```

```
        if (AlertUtil.showConfirmation("Logout", "Are you sure you want to logout?")) {
            MainApp.getInstance().setCurrentUsername(null);
            ApiService.getInstance().setCurrentUsername(null);
            SceneSwitcher.switchTo("login");
        }
    }
}
```

ApiService.java

```
package com.securevaultclient.service;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import com.securevaultclient.model.AuditLogModel;
import com.securevaultclient.model.FileMetadataModel;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.List;

public class ApiService {

    private static final String BASE_URL = "http://localhost:8081/api";
    private static ApiService instance;
    private final HttpClient httpClient;
    private final ObjectMapper objectMapper;
    private String currentUsername;
```

```
    private ApiService() {
        this.httpClient = HttpClient.newHttpClient();
        this.objectMapper = new ObjectMapper();
        this.objectMapper.registerModule(new JavaTimeModule());
        this.objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES,
false);
    }

    public static ApiService getInstance() {
        if (instance == null) {
            instance = new ApiService();
        }
        return instance;
    }

    public void setCurrentUsername(String username) {
        this.currentUsername = username;
    }

    public FileMetadataModel addFile(String fileName, String fileType, Long size, String owner)
            throws Exception {
        String json = objectMapper.writeValueAsString(
            new FileMetadataModel(null, fileName, owner, fileType, size, null, null, null)
        );

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + "/files/add"))
            .header("Content-Type", "application/json")
            .header("username", currentUsername)
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();
```

```
    HttpResponse<String> response = httpClient.send(request,
        HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() == 200) {
        return objectMapper.readValue(response.body(), FileMetadataModel.class);
    } else {
        throw new RuntimeException("Error adding file: " + response.body());
    }
}

public List<FileMetadataModel> getAllFiles() throws Exception {
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(BASE_URL + "/files/all"))
        .header("username", currentUsername)
        .GET()
        .build();

    HttpResponse<String> response = httpClient.send(request,
        HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() == 200) {
        return objectMapper.readValue(response.body(),
            new TypeReference<List<FileMetadataModel>>() {});
    } else {
        throw new RuntimeException("Error getting files: " + response.body());
    }
}

public String verifyFile(Long fileId) throws Exception {
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(BASE_URL + "/files/verify/" + fileId))
        .header("Content-Type", "application/json")
```

```
        .header("username", currentUsername)
        .POST(HttpRequest.BodyPublishers.noBody())
        .build();

    HttpResponse<String> response = httpClient.send(request,
        HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() == 200) {
        var jsonNode = objectMapper.readTree(response.body());
        return jsonNode.get("status").asText() + ": " + jsonNode.get("message").asText();
    } else {
        throw new RuntimeException("Error verifying file: " + response.body());
    }
}

// Fixed: use HttpClient instead of HttpURLConnection
public String callServlet() {
    try {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("http://localhost:8081/hello"))
            .GET()
            .build();

        HttpResponse<String> response = httpClient.send(request,
            HttpResponse.BodyHandlers.ofString());

        return response.body();
    } catch (Exception e) {
        return "Error: " + e.getMessage();
    }
}
```

```
    public List<AuditLogModel> getAllAuditLogs() throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + "/audit/all"))
            .header("username", currentUsername)
            .GET()
            .build();

        HttpResponse<String> response = httpClient.send(request,
            HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() == 200) {
            return objectMapper.readValue(response.body(),
                new TypeReference<List<AuditLogModel>>() {});
        } else {
            throw new RuntimeException("Error getting audit logs: " + response.body());
        }
    }
}
```

dashboard.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.BorderPane?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
        fx:controller="com.securevaultclient.controller.DashboardController">
```

```
<top>
    <VBox alignment="CENTER" spacing="10" style="-fx-background-color: #2c3e50;">
        <padding>
            <Insets bottom="15" left="15" right="15" top="15" />
        </padding>

        <Label fx:id="welcomeLabel" text="Welcome!" textFill="white">
            <font>
                <Font name="System Bold" size="20" />
            </font>
        </Label>
    </VBox>
</top>

<center>
    <GridPane alignment="CENTER" hgap="20" vgap="20">
        <padding>
            <Insets bottom="30" left="30" right="30" top="30" />
        </padding>

        <!-- Row 0 -->
        <Button fx:id="addFileBtn" mnemonicParsing="false" onAction="#handleAddFile"
                prefHeight="100" prefWidth="150"
                style="-fx-background-color: #27ae60; -fx-text-fill: white;"
                text="Add File" GridPane.rowIndex="0" GridPane.columnIndex="0">
            <font><Font size="14" /></font>
        </Button>

        <Button fx:id="viewFilesBtn" mnemonicParsing="false" onAction="#handleViewFiles"
                prefHeight="100" prefWidth="150"
                style="-fx-background-color: #2980b9; -fx-text-fill: white;"
```

```
        text="View Files" GridPane.rowIndex="0" GridPane.columnIndex="1">
    <font><Font size="14" /></font>
</Button>

<!-- Row 1 -->
<Button fx:id="verifyBtn" mnemonicParsing="false" onAction="#handleVerifyFile"
        prefHeight="100" prefWidth="150"
        style="-fx-background-color: #f39c12; -fx-text-fill: white;"
        text="Verify File" GridPane.rowIndex="1" GridPane.columnIndex="0">
    <font><Font size="14" /></font>
</Button>

<Button fx:id="auditBtn" mnemonicParsing="false" onAction="#handleViewAudit"
        prefHeight="100" prefWidth="150"
        style="-fx-background-color: #8e44ad; -fx-text-fill: white;"
        text="View Audit" GridPane.rowIndex="1" GridPane.columnIndex="1">
    <font><Font size="14" /></font>
</Button>

<!-- Row 2 -->
<Button fx:id="servletBtn" mnemonicParsing="false" onAction="#handleServlet"
        prefHeight="100" prefWidth="150"
        style="-fx-background-color: #16a085; -fx-text-fill: white;"
        text="Call Servlet" GridPane.rowIndex="2" GridPane.columnIndex="0">
    <font><Font size="14" /></font>
</Button>

<!-- Row 3 -->
<Button fx:id="logoutBtn" mnemonicParsing="false" onAction="#handleLogout"
        prefHeight="50" prefWidth="320"
        style="-fx-background-color: #e74c3c; -fx-text-fill: white;"
        text="Logout" GridPane.columnSpan="2" GridPane.rowIndex="3">
```

```
            <font><Font size="14" /></font>
        </Button>
    </GridPane>
  </center>
</BorderPane>
```

Servlet Code

HelloServlet.java

```
package com.securevault;

import jakarta.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.*;

public class HelloServlet extends HttpServlet {

  @Override
  protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

    HttpSession session = request.getSession();
    Integer count = (Integer) session.getAttribute("count");
    if (count == null) count = 1;
    else count++;
    session.setAttribute("count", count);

    System.out.println("<p>Visited " + count + " times</p>");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
```

```
    out.println("<h2>File Metadata (From Database)</h2>");

    try {
        Connection conn = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/securevault",
            "root",
            "114114"
        );

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM file_metadata");

        out.println("<table border='1'>");
        out.println("<tr><th>ID</th><th>Name</th><th>Owner</th><th>Type</th></tr>");

        while (rs.next()) {
            out.println("<tr>");
            out.println("<td>" + rs.getLong("id") + "</td>");
            out.println("<td>" + rs.getString("file_name") + "</td>");
            out.println("<td>" + rs.getString("owner") + "</td>");
            out.println("<td>" + rs.getString("file_type") + "</td>");
            out.println("</tr>");
        }

        out.println("</table>");

        conn.close();

    } catch (SQLException e) {
        out.println("<p>Error: " + e.getMessage() + "</p>");
    }
}
```

```
}
```

Backend Code

SecureVaultApplication.java

```
package com.securevault;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.boot.web.servlet.ServletRegistrationBean;
import org.springframework.context.annotation.Bean;




@SpringBootApplication
public class SecureVaultApplication {
    @Bean
    public ServletRegistrationBean<HelloServlet> servletBean() {
        return new ServletRegistrationBean<>(new HelloServlet(), "/hello");
    }
    public static void main(String[] args) {
        SpringApplication.run(SecureVaultApplication.class, args);
    }
}
```

VaultController.java

```
package com.securevault.controller;

import com.securevault.dto.FileRequestDTO;
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.AuditLog;
```

```
import com.securevault.service.AuditService;
import com.securevault.service.VaultService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")
public class VaultController {

    @Autowired
    private VaultService vaultService;

    @Autowired
    private AuditService auditService;

    @PostMapping("/files/add")
    public ResponseEntity<FileResponseDTO> addFile(@RequestBody FileRequestDTO request,
                                                   @RequestHeader("username") String username) {
        FileResponseDTO response = vaultService.addFile(request, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/files/all")
    public ResponseEntity<List<FileResponseDTO>> getAllFiles() {
        List<FileResponseDTO> files = vaultService.getAllFiles();
        return ResponseEntity.ok(files);
    }

    @PostMapping("/files/verify/{id}")
```

```
    public ResponseEntity<VerifyResponseDTO> verifyFile(@PathVariable Long id,
                                                        @RequestHeader("username") String username) {
        VerifyResponseDTO response = vaultService.verifyFile(id, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/audit/all")
    public ResponseEntity<List<AuditLog>> getAllAuditLogs() {
        List<AuditLog> logs = auditService.getAllAuditLogs();
        return ResponseEntity.ok(logs);
    }
}
```

VaultService.java

```
package com.securevault.service;

import com.securevault.dto.FileRequestDTO;
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.FileMetadata;
import com.securevault.repository.FileMetadataRepository;
import com.securevault.util.DateUtil;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class VaultService {

    @Autowired
```

```
private FileMetadataRepository fileMetadataRepository;

@Autowired
private HashService hashService;

@Autowired
private RiskAnalysisService riskAnalysisService;

@Autowired
private AuditService auditService;

@Transactional
public FileResponseDTO addFile(FileRequestDTO request, String username) {
    // Generate hash
    String hash = hashService.generateFileHash(
        request.getFileName(),
        request.getOwner(),
        request.getSize()
    );

    // Analyze risk
    String riskLevel = riskAnalysisService.analyzeRiskLevel(request.getFileName());

    // Create file metadata
    FileMetadata metadata = new FileMetadata(
        request.getFileName(),
        request.getOwner(),
        request.getFileType(),
        request.getSize(),
        DateUtil.getCurrentDateTime(),
        hash,
        riskLevel
```

```
    );

    FileMetadata saved = fileMetadataRepository.save(metadata);

    // Log action
    auditService.logAction(username, "Added file: " + request.getFileName() +
        " (Risk: " + riskLevel + ")");

    return convertToDTO(saved);
}

public List<FileResponseDTO> getAllFiles() {
    return fileMetadataRepository.findAll().stream()
        .map(this::convertToDTO)
        .collect(Collectors.toList());
}

public VerifyResponseDTO verifyFile(Long fileId, String username) {
    FileMetadata file = fileMetadataRepository.findById(fileId)
        .orElseThrow(() -> new RuntimeException("File not found with id: " + fileId));

    // Recalculate hash using the same method
    String currentHash = hashService.generateFileHash(
        file.getFileName(),
        file.getOwner(),
        file.getSize()
    );

    String status = currentHash.equals(file.getHashValue()) ? "VERIFIED" : "TAMPERED";
    String message = status.equals("VERIFIED") ?
        "File integrity is intact" :
        "File has been modified!";
```

```
        // Log action
        auditService.logAction(username, "Verified file: " + file.getFileName() +
            " - Status: " + status);

        return new VerifyResponseDTO(
            fileId,
            file.getFileName(),
            file.getHashValue(),
            currentHash,
            status,
            message
        );
    }

    private FileResponseDTO convertToDTO(FileMetadata metadata) {
        return new FileResponseDTO(
            metadata.getId(),
            metadata.getFileName(),
            metadata.getOwner(),
            metadata.getFileType(),
            metadata.getSize(),
            metadata.getUploadDate(),
            metadata.getHashValue(),
            metadata.getRiskLevel()
        );
    }
}
```

FileMetadataRepository.java

```
package com.securevault.repository;
```

```
import com.securevault.model.FileMetadata;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface FileMetadataRepository extends JpaRepository<FileMetadata, Long> {
    List<FileMetadata> findByOwner(String owner);
    List<FileMetadata> findByRiskLevel(String riskLevel);
}
```

Database

application.properties

```
# Server Configuration
server.port=8081

# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/securevault?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=114114
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.properties.hibernate.format_sql=true

# Logging
logging.level.org.springframework.web=INFO
logging.level.com.securevault=DEBUG
```

HTML File

index.html

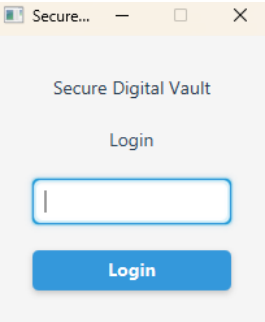
```
<!DOCTYPE html>
<html>
<head>
    <title>25MCA1072 | Servlet</title>
</head>
<body>

<h2>View File Data (Servlet + JDBC)</h2>

<form action="/hello" method="get">
    <button type="submit">Load Files from Database</button>
</form>

</body>
</html>
```

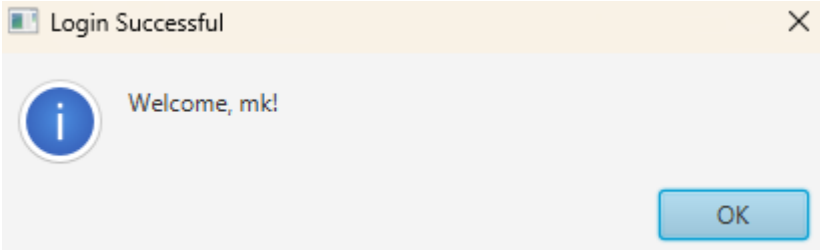
Execution Screenshots

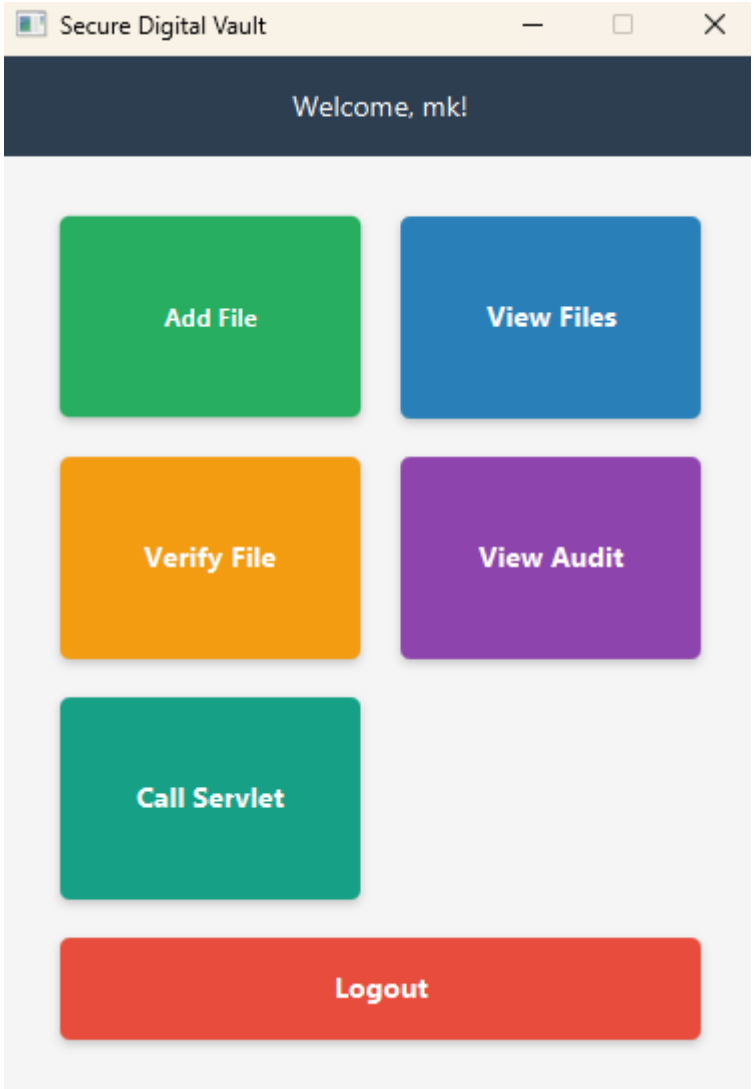
SpringBoot Running

```
--- resources:3.3.1:resources (default-resources) @ secure-vault-server ---
Copying 1 resource from src\main\resources to target\classes
Copying 4 resources from src\main\resources to target\classes

--- compiler:3.11.0:compile (default-compile) @ secure-vault-server ---
Nothing to compile - all classes are up to date

--- exec:3.5.1:exec (default-cli) @ secure-vault-server ---

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.1.5)

2026-03-25T00:02:41.962+05:30  INFO 28948 --- [           main] com.securevault.SecureVaultApplication   : Starting SecureVaultApplication using Java 25.0.1 with PID 28948 (C:\SEM 2\JAVA
2026-03-25T00:02:41.966+05:30 DEBUG 28948 --- [           main] com.securevault.SecureVaultApplication   : Running with Spring Boot v3.1.5, Spring v6.0.13
2026-03-25T00:02:41.968+05:30  INFO 28948 --- [           main] com.securevault.SecureVaultApplication   : No active profile set, falling back to 1 default profile: "default"
2026-03-25T00:02:43.509+05:30  INFO 28948 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-03-25T00:02:43.665+05:30  INFO 28948 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 134 ms. Found 3 JPA repository inte
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.apache.tomcat.jni.Library in an unnamed module (file:/C:/Users/fallen/.m2/repository/org/apache/tomcat/embed/tomcat-embed-core/10.
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

2026-03-25T00:02:44.967+05:30  INFO 28948 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8081 (http)
2026-03-25T00:02:44.995+05:30  INFO 28948 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-03-25T00:02:44.996+05:30  INFO 28948 --- [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.15]
2026-03-25T00:02:45.164+05:30  INFO 28948 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
```

JavaFX UI(Login/Dashboard)







HTML Page & Servlet Output:

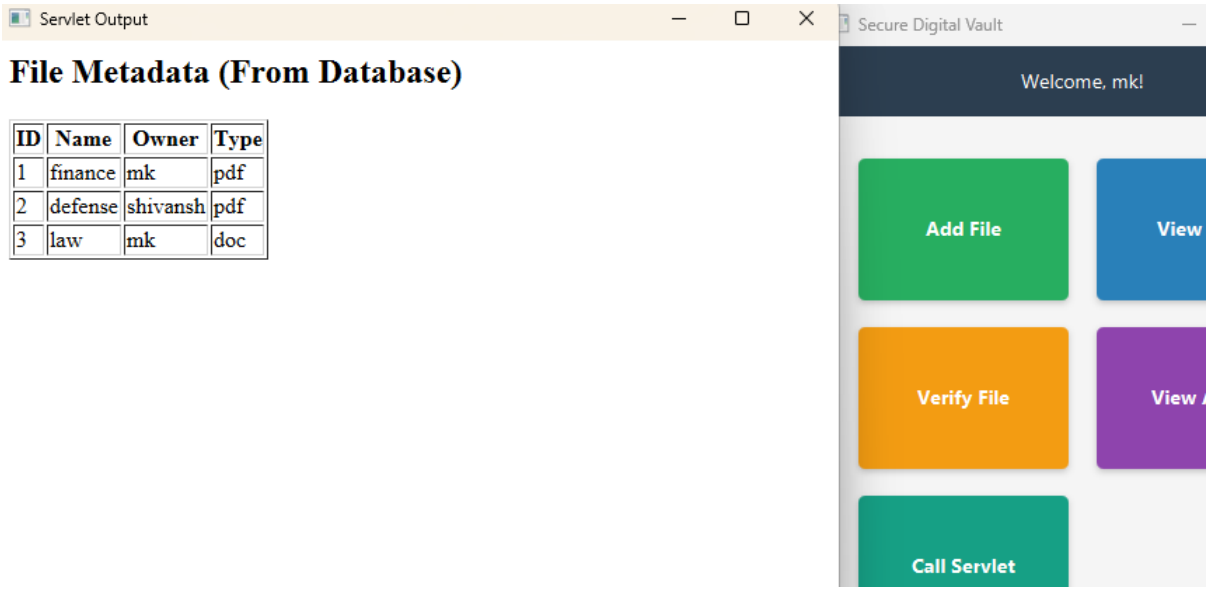
View File Data (Servlet + JDBC)

Load Files from Database

File Metadata (From Database)

ID	Name	Owner	Type
1	finance	mk	pdf
2	defense	shivansh	pdf
3	law	mk	doc

JavaFX WebView rendering servlet output



MySQL Database Table

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	file_name	file_type	hash_value	owner	risk_level	size	upload_date
1	finance	pdf	ed8741c19facad5a74d4ab85bbb6f9a00bbc095...	mk	LOW	1024	2026-03-01 09:40:52.459696
2	defense	pdf	843c703530e884e1c348bccbe60b0dcb7003b6e...	shivansh	LOW	1056	2026-03-01 18:16:41.375874
3	law	doc	0d06e0f7b37dd3d3f190c44717286a7d5059563...	mk	LOW	1024	2026-03-24 18:52:35.027894
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Current Status

The project is approximately 60–70% complete. The backend and database integration are fully functional. The JavaFX front-end is connected to the backend and capable of displaying data. The servlet has been successfully implemented with session management and database interaction. Integration between all components is achieved.

Summary

The project demonstrates a complete Java-based application using JavaFX for frontend, Spring Boot for backend, MySQL with JDBC for database operations, and servlet implementation for request handling and session management. It follows MVC architecture and successfully integrates multiple technologies into a unified system.